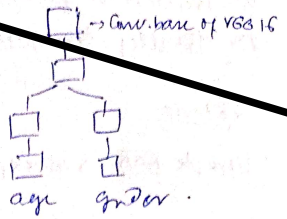
Multi output Model

age/emotion → [CNN] → 2000 → age, gender.

Transfer learning

VGG16 → Conv. Base.



Sequence models

RNN — Recurrent Neural Network

ANN → tabular data
CNN → images/grid.

RNN → is a type of sequential model to work on sequential data

iq marks gender placement ?

Sequence matters-> order of the input matters

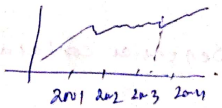
Non sequential data.
if iq & marks position changed placement do not affect.

text data

Hi My name is Anik ✓

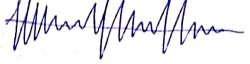
Hi Anik is my name. ✗

Time series



DNA sequence

Speech



OHE is ok but not efficeient use EMbedding

Why use RNN?

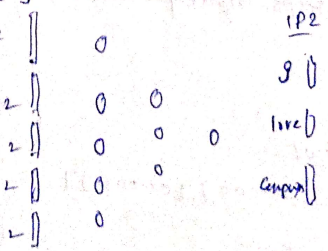
ip	op	op
Hi My name is Anik (5)		0
I love complex (3)		0
India won the match (4)		1

(12) Vectorize (OHE)

[1,0,0,...0] [0,1,0,...0] [0,0,1,0,...0] ... [0,0,0,0,1,...0]

Hi My name Anik

ANN → text classification



No fixed I/p size

Hi input layers vary

↓

different ip size

ANN process input independently
ANN cant retain information (no memory) about order of i/ps so we need RNN
RNN process data one step at a time, maintaining hidden state(memory)->for future steps

Problems.

1) Text input → varying size
2) Zero padding → unnecessary computation
3) Prediction problem.
4) Totally disregarding the sequence information.

inefficient to handlle variable i/p

Roadmap

Simple RNN → Backpropagation RNN. (Backprop in time) → vGP, EGP, → Solve → LSTM GRU.

Types of RNN

Bidirectional RNN

Deep RNN

Seq Seq Model

Application
Sentiment analysis
sentence completion
image captioning
machine translation
question answering

1/08/25

Why RNN?

1) Sequence → any length → [ANN] → fixed ip size

movie review

Zero padding → cost of computation

2) Sequence contains some meaning.

RNN
ANN
(memory feature)

Eg: Sentement Analysis

Data for RNN

(timesteps, input-features)

Review	Sentiment
Movie was good	1
movie was bad	0
movie was not good	0

review1

[[10000],[01000],[00100]]

(3,5)

No. of timestamps.

No. of features

review2 → (3,5)

review3 → (4,5)

Vocab → 5 word.

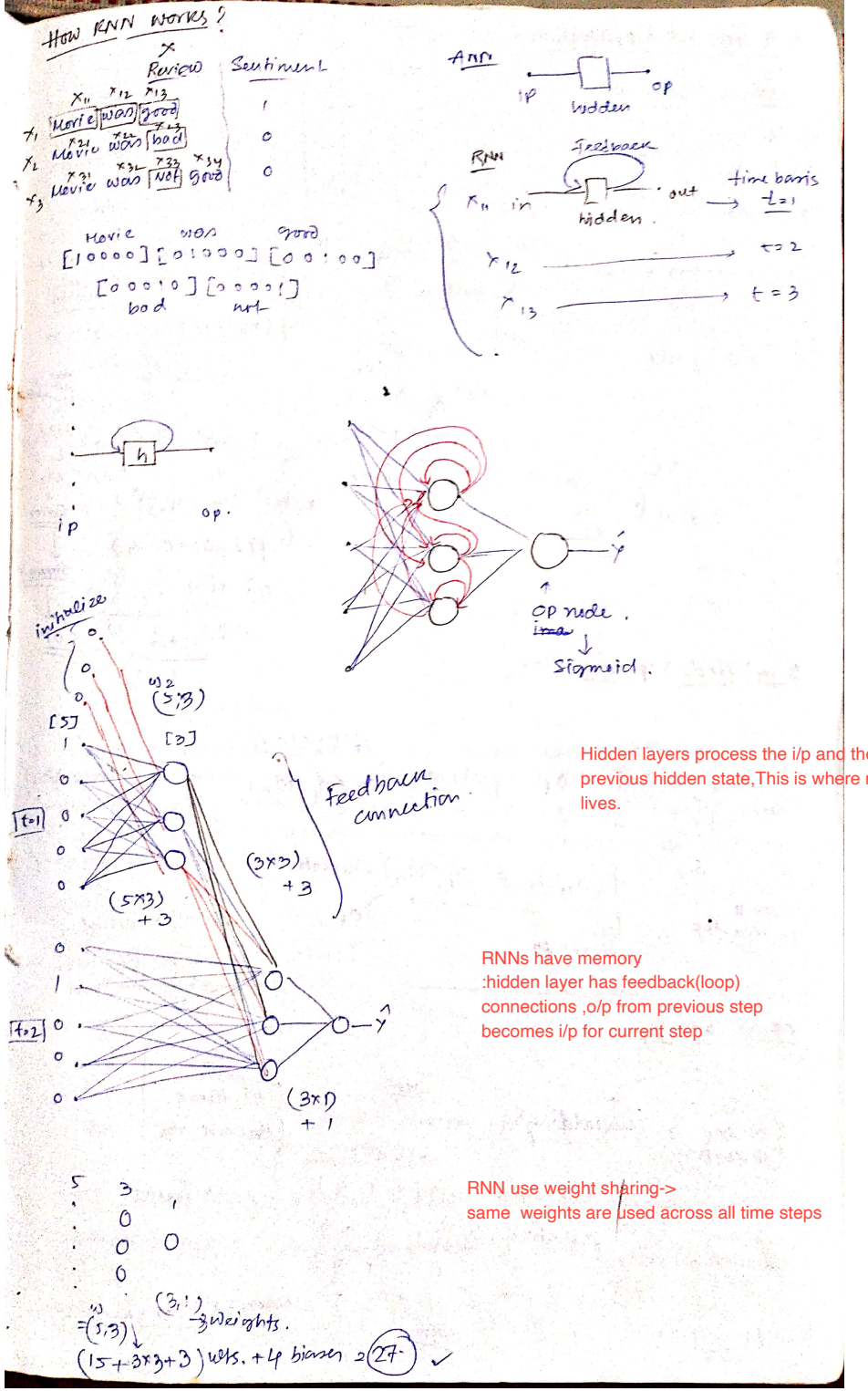
Movie [1 0 0 0 0] was [0 1 0 0 0] good [0 0 1 0 0]

bad [0 0 0 1 0] not [0 0 0 0 1]

Keras → Simple CNN → (batch size, timestep, input feature)

Scanned with OKEN Scanner

How RNN Works?

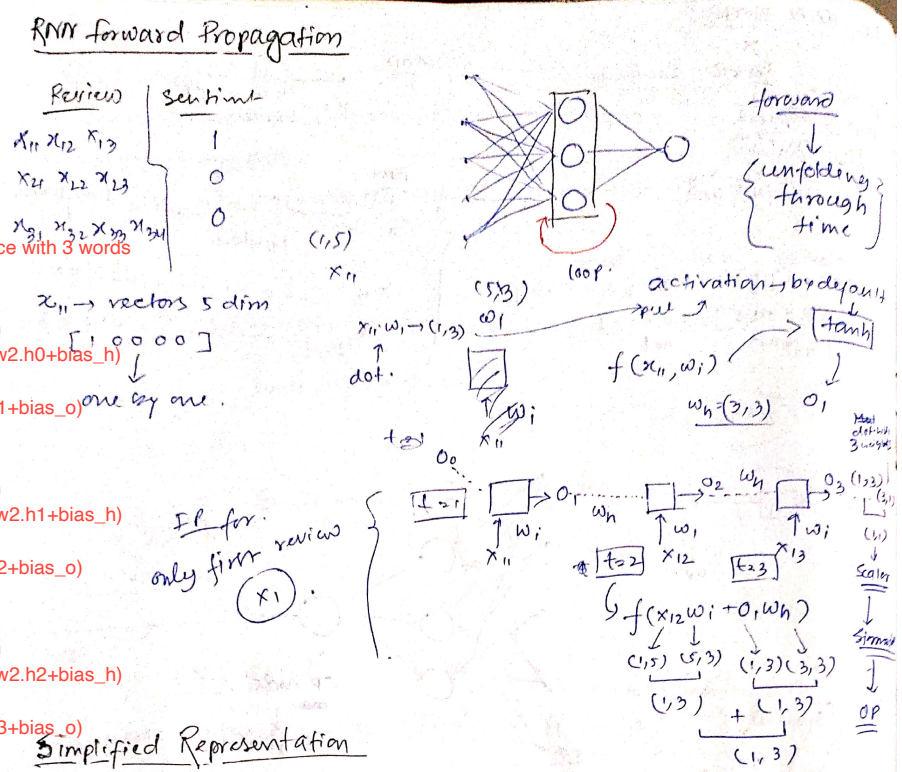


Hidden layers process the i/p and the previous hidden state,This is where memory lives.

RNNs have memory
:hidden layer has feedback(loop) connections ,o/p from previous step becomes i/p for current step

RNN use weight sharing->
same weights are used across all time steps

RNN forward Propagation



Input for 1 sentence with 3 words
[X11,X12,X13]
at t=1,
hidden layer calcn
h1=tanh(w1.x11+w2.h0+bias_h)
o/p calcn
O1=sigmoid(w3.h1+bias_o)

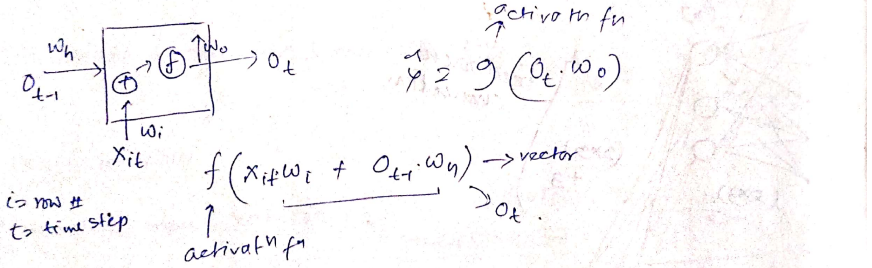
at t=2
hidden layer calcn
h2=tanh(w2.x12+w2.h1+bias_h)
o/p calcn
O2=sigmoid(w3.h2+bias_o)

at t=3
hidden layer calcn
h3=tanh(w2.x13+w2.h2+bias_h)
o/p calcn
O3=sigmoid(w3.h3+bias_o)

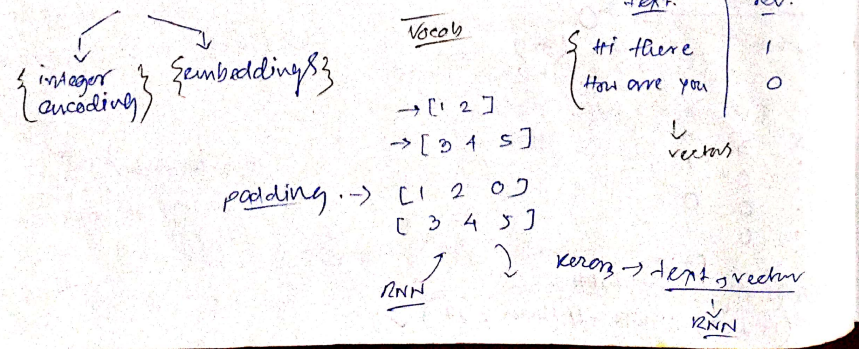
$x_{11} \rightarrow$ vectors 5 dim
[1 0 0 0 0]
one by one.

IP for only first review (x_1)

Simplified Representation



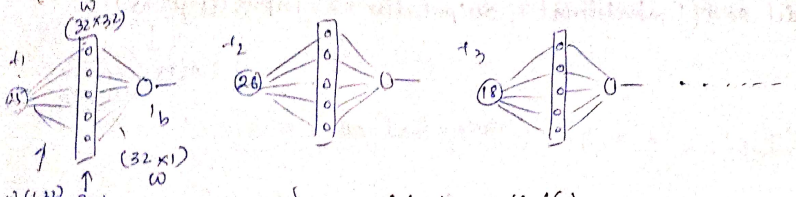
Keras Code Eg.



Scanned with OKEN Scanner

50 numbers → 50 timestep ($t_1, t_2, \dots t_{50}$)

[25, 26, 18, ...]



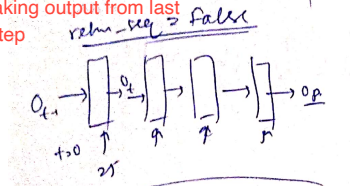
$32 + 32\times32 + 32 + 32\times1$

$= 1141$

```
model = Sequential()
model.add(SimpleRNN(32, input_shape = (50,1), return_sequences = False))
model.add(Dense(1, activation = 'sigmoid'))
model.summary()
```

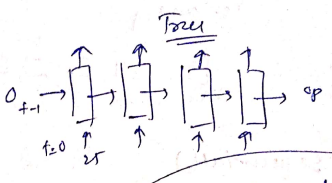
Mostly, return sequence=False only taking output from last time step

return_seq = False



only get the last output

return_seq = True



getall the outputs in every step.

Embedding...

In NLP, word embedding is a term used for the representation of words, for text analysis, typically in the form of a real-valued vector that encodes the meaning of the word such that the words that are closer in the vector space are expected to be similar in meaning.

review1 → 20000 → [.....]

review2 → 20000 [.....]

20 → 1980 0's

[.....]

(Sparse vector) ← 2000 words → 1980 0's

Sparse representation: [0.7 0 0.1 0.3] (this is nice)

(dense) non-zero.

word Embedding → dense vector; semantic meaning (dim↓)

integer encoding → Sparse vector

word → context

Better Result

dim↓

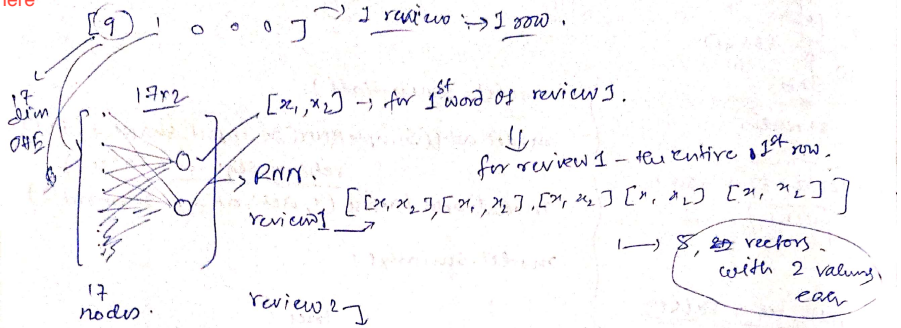
Weight matrix shape (vocabulary_size,embedding_dimension)

```
model = Sequential()
(len(tokenizer.word_index) → no. of unique words.)
model.add(Embedding(17, output_dim = 2, input_length = 25))
```

max sentence length.

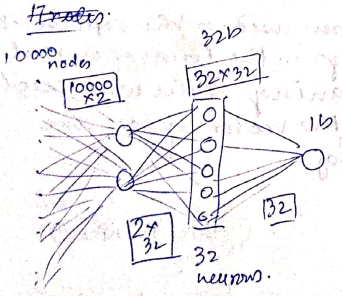
Meaning here

[9 1 0 0 0] → 1 review → 1 row.



for review 1 – the entire 1st row.

```
model = Sequential()
model.add(Embedding(10000, output_dim = 2, input_length = 50))
model.add(SimpleRNN(32, return_sequence = False))
model.add(Dense(1, activation = 'sigmoid'))
```



10000 dim

$20000 + 64 + 32\times32 + 32 + 32 + 1$

Types of RNN

- Many to one
- one to Many
- Many to Many
- One to Many

depends upon I/p and o/p type (sequential (mutiple time step) or non-sequential (single timestep))

Architecture 1

Many to One

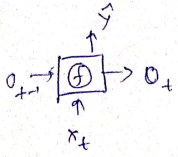
step by step words in sentences

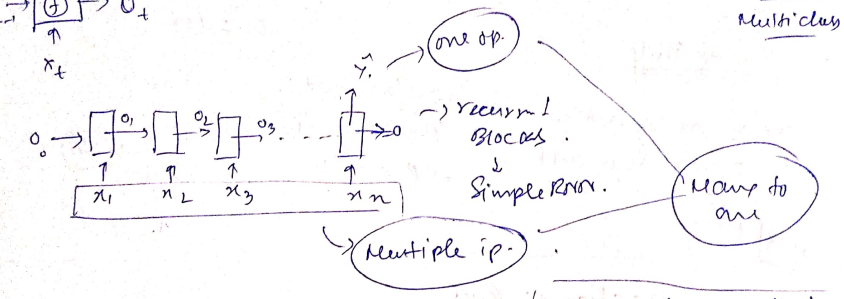
input → Sequence → sentence, char, time series data

output → non-sequence → int, real no. → scalar (1, 0) ... single value

→ Sentiment Analysis → seq. → 1, 0

→ Rating Prediction → review → 1 → 5 ⋆ ⋆ ⋆ ⋆ ... ∈ (1, 5) Multi class





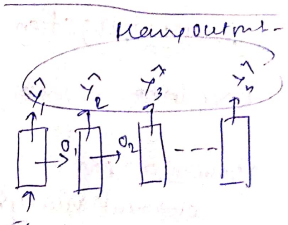
Architecture 2

One to Many

→ normal non-sequential → → op → sequential.

→ Image Captioning → ip → img → op → textual representation (A scene of sunset) seq. data.

non seq.



→ Music generation → ....

Architecture 3

Many to Many

ip → sequential
op → sequential
→ Seq to Seq.

Many to many RNN has two types

→ Same length
input seq = = output seq.
→ POS tagging → NLP → (Chatbots)

→ Variable length

every input word -> every word labelling as output

Variable length

machine translation

→ lang1 → lang2

Google translate

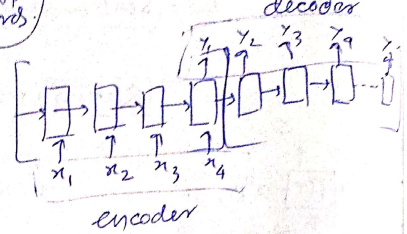
Many to many RNN-. variable length is also called encoder-decoder where encoder reads the i/p and decoder generates the o/p.

{encoder decoder}

Hi my name is Anik.

diff no. of words

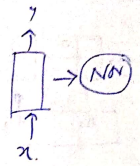
मेरा नाम अनिक है



One to One

One to One is not an RNN it becomes ANN or CNN

one → non sequential
one → non sequential → img classification. ip → img. op → 0/1



06/08/25

Backpropagation of RNN.

RNN ~~Intro~~ → Backpropagation → BPTT

Many to one RNN

sentiment analysis
text → 1/0.

this or add embedding layer in RNN which will handle it itself

text	Sentiment
cat mat rat	1
rat rat mat	1
mat mat cat	0

x

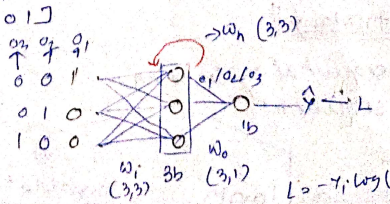
x_1 [1 0 0] [0 1 0] [0 0 1]
x_2 [0 0 1] [0 0 1] [0 1 0]
x_3 [0 1 0] [0 1 0] [1 0 0]

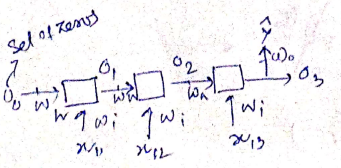
y

1
1
0

vocab → cat mat rat.
[1 0 0] [0 1 0] [0 0 1]

forward propagation.





$o_1 = f(x_{11} w_i + o_0 w_h)$

$o_2 = f(x_{12} w_i + o_1 w_h)$

$o_3 = f(x_{13} w_i + o_2 w_h)$

$\hat{y} = \sigma(o_3 \cdot w_o)$

$L = -y_i \log(\hat{y}_i) - (1-y_i) \log(1-\hat{y}_i)$

loss calculate → gradient descent → minimize

$w_i \quad w_h \quad w_o$

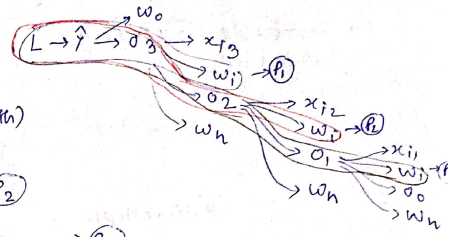
$w_i = w_i - \eta \left[\frac{\partial L}{\partial w_i}\right]$

$w_h = w_h - \eta \cdot \left[\frac{\partial L}{\partial w_h}\right]$ Backprop.

$w_o = w_o - \eta \left[\frac{\partial L}{\partial w_o}\right]$

$L \to \hat{y} \to o_3 \to w_o$

$$\frac{\partial L}{\partial w_o} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_o}$$



$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial w_i} \Rightarrow P_1 \text{ (Born)}$$

$$+ \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial o_2} \cdot \frac{\partial o_2}{\partial w_i} \Rightarrow P_2$$

$$+ \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial o_2} \cdot \frac{\partial o_2}{\partial o_1} \cdot \frac{\partial o_1}{\partial w_i} \Rightarrow P_3$$

$$\frac{\partial L}{\partial w_i} = \sum_{j=1}^{3} \left(\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_j} \cdot \frac{\partial o_j}{\partial w_i}\right) \quad t=1 = \frac{\partial L}{\partial \hat{y}} \left[\frac{\partial \hat{y}}{\partial o_1}\right] \frac{\partial o_1}{\partial w_i}$$

$$\left\{\frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial o_2} \cdot \frac{\partial o_2}{\partial o_1}\right\}$$

$$t=2 = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_2} \cdot \frac{\partial o_2}{\partial w_i} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial o_2} \cdot \frac{\partial o_2}{\partial w_i}$$

$$t=3 = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial w_i} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial w_i}$$

$$\frac{\partial L}{\partial w_i} = \sum_{j=1}^{n} \left(\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_j} \cdot \frac{\partial o_j}{\partial w_i}\right) \quad \{n = \#\text{timesteps}\}$$

$$\frac{\partial L}{\partial w_h} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial w_h} + \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial o_2} \cdot \frac{\partial o_2}{\partial w_h} + \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial o_2} \cdot \frac{\partial o_2}{\partial o_1} \cdot \frac{\partial o_1}{\partial w_h}$$

$$\frac{\partial L}{\partial w_h} = \sum_{j=1}^{n} \left(\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_j} \cdot \frac{\partial o_j}{\partial w_h}\right) \quad \{n = \#\text{timesteps}\}$$

Problems with RNN

RNNs → sequential data → textual, time series.

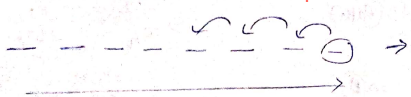
↓

Suffer (2) major problem → LSTM why?

Due to VGP → Problem of long term dependency. } RNN ✗

Due to EGP → ~~Unstable~~ Training: Stagnated

new data depends on old data



Applicatn: next word predicting

→ Marathi is spoken in Maharashtra

↳ short term dependency

the sentence is short.

RNN will fail on long sentences

Maharashtra is a beautiful place. I went there last year. But I couldn't enjoy properly because I don't understand Marathi.

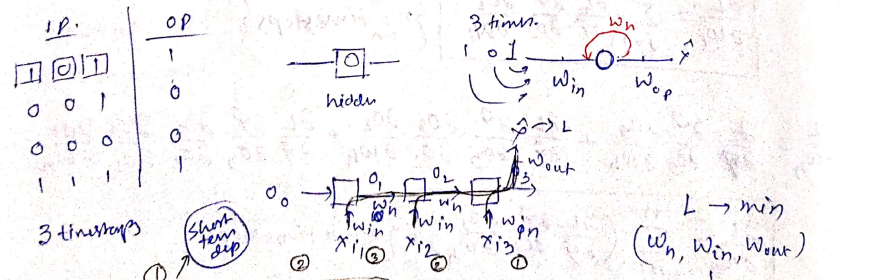
↳ long-term dependency.

{Vanishing Gradient problem.} RNN ✗

↳ Long term dependency problem in RNN.

Problem #1 → Problem of Long Term Dependency. → VGP.

Long term Dependency is not about memory but gradient math



$$\frac{\partial L}{\partial w_{in}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial w_{in}} + \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial o_2} \cdot \frac{\partial o_2}{\partial w_{in}} + \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_3} \cdot \frac{\partial o_3}{\partial o_2} \cdot \frac{\partial o_1}{\partial o_1} \cdot \frac{\partial o_1}{\partial w_{in}}$$

for 3 timesteps.

$L \to \min(w_h, w_{in}, w_{out})$

↓ gradient descent

$$w_{in} = w_{in} - \eta \frac{\partial L}{\partial w_{in}}$$

$$w_{out} = w_{out} - \eta \frac{\partial L}{\partial w_{out}}$$

$$w_h = w_h - \eta \cdot \frac{\partial L}{\partial w_h}$$

for 100th timestep → last term

$$\frac{\partial L}{\partial \hat{y}^{\vec{}}} \cdot \frac{\partial \hat{y}^{\vec{}}}{\partial o_{100}} \cdot \frac{\partial o_{100}}{\partial o_{99}} \cdot \cdots \cdot \frac{\partial o_2}{\partial o_1} \cdot \frac{\partial o_1}{\partial w_{in}}$$ → longterm dep

$$\frac{\partial L}{\partial \hat{y}^{\vec{}}} \cdot \frac{\partial \hat{y}^{\vec{}}}{\partial o_{100}} \cdot \prod_{t=2}^{100} \left(\frac{\partial o_t}{\partial o_{t-1}}\right) \frac{\partial o_1}{\partial w_{in}}$$

$$\left\{ \frac{\partial o_2}{\partial o_1} \cdot \frac{\partial o_3}{\partial o_2} \cdot \frac{\partial o_4}{\partial o_3} \cdots \frac{\partial o_{100}}{\partial o_{99}} \right\}$$

$o_1 = \tanh(x_{i_1} w_{in} + o_0 w_h)$

$o_t = \tanh(x_{i_t} w_{in} + o_{t-1} w_h)$

$\frac{\partial o_t}{\partial o_{t-1}} = \tanh'(x_{i_t} w_{in} + o_{t-1} w_h) \cdot w_h$

$\tanh' \in (0,1)$

Derivative of tanh is 0 to 1

$$\frac{\partial L}{\partial \hat{y}^{\vec{}}} \cdot \frac{\partial \hat{y}^{\vec{}}}{\partial o_{100}} \prod_{t=2}^{100} \left(\frac{\partial o_t}{\partial o_{t-1}}\right) \frac{\partial o_1}{\partial w_{in}}$$

$$= \frac{\partial L}{\partial \hat{y}^{\vec{}}} \cdot \frac{\partial \hat{y}^{\vec{}}}{\partial o_{100}} \prod_{t=2}^{100} \left(\tanh'(\,) \cdot w_h\right) \cdot \frac{\partial o_1}{\partial w_{in}}$$

$\tanh' \in (0,1)$, $w_h \in (0,1)$

Suppose $w_h \in (0,1)$

VGP

→ very small no. $\simeq 0$ $\simeq 0$.

↳ Long term dependency $\simeq 0$ → all contribution shift towards short term dependency.

how to reduce this problem ??

1) Diff activation → relu / leaky relu
2) Better weight init.
3) Skip RNNs.
4) LSTM architecture.

Problem #2 → Stagnated Training (EGP)

Suppose you used Relu instead of tanh

↓

the gradients can be very large with too much multiplication.

$\frac{\partial L}{\partial w_i}$, $\frac{\partial L}{\partial w_h}$ → can be very large.

↓

Training Step.

→ OR, can be the LR (η) very large

Scanned with OKEN Scanner